

KTH

DEGREE PROJECT IN COMPUTER SCIENCE AND ENGINEERING
SECOND CYCLE, 30 CREDITS

Stockholm, Sweden 2026

Joint Radar Emitter and Mode Clustering Using a Transformer Encoder Architecture

Markus Swegmark

Supervisor: TBD

Examiner: TBD

Host: TBD

KTH Royal Institute of Technology
School of Electrical Engineering and Computer Science
Master's Programme in Machine Learning

TRITA-EECS-EX-XXXX:XXX

Abstract

This thesis investigates joint clustering of radar emitters and their operating modes from intercepted pulse descriptor word (PDW) streams using a transformer encoder architecture.

Keywords: transformer, deep clustering, electronic intelligence, radar emitter identification, mode recognition, self-supervised learning.

Sammanfattning

Detta examensarbete undersöker gemensam klustering av radarsändare och deras operativa moder från avlyssnade pulsbeskrivande ord (PDW) med hjälp av en transformerbaserad encoderarkitektur.

Nyckelord: transformer, djup klustering, signalspaning, radarsändaridentifiering, modigenkänning, självövervakad inlärning.

Acknowledgments

I would like to thank my supervisor and examiner at KTH, and the host organization for their support throughout this thesis project.

Stockholm, May 2026

Markus Swegmark

Contents

Abstract	i
Sammanfattning	ii
Acknowledgments	iii
List of Acronyms	ix
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Research Questions	2
1.4 Objectives and Scope	2
1.4.1 Objectives	2
1.4.2 Delimitations	2
1.5 Contributions	3
1.6 Ethical and Societal Considerations	3
1.7 Thesis Outline	4
2 Background	5
2.1 Radar Systems and Passive Reception	5
2.1.1 Electromagnetic Pulses for navigation, surveillance, and control	5
2.1.2 Time-of-Flight Ranging and Ambiguity Fundamentals	5
2.1.3 Transmitting radars versus receive-only systems	6
2.1.4 From passband samples to a list of pulses	6
2.1.5 Pulse descriptor words and the pulse train	7
2.1.6 Physical sources and transmit schedules	8
2.2 Signal Processing Pipeline for Passive ELINT	8
2.2.1 Front-end reception, detection, and pulse-parameter estimation .	9
2.2.2 Deinterleaving and pulse-train formation	9
2.2.3 Emitter reporting and equipment identification	10
2.2.4 Operating-mode analysis and waveform libraries	10
2.2.5 Emitter geolocation, fusion, and downstream displays	10
2.3 Machine Learning Fundamentals	10
2.4 Clustering Methods	11
2.5 Deep Representation Learning	12
2.6 The Transformer Encoder	13

3	Related Work	15
3.1	Classical Radar Emitter Identification	15
3.2	Deep Learning for Radar Signal Analysis	15
3.3	Deep Clustering and Joint Representation Learning	15
3.4	Transformers for Time Series and Sets	16
3.5	Joint and Multi-Task Clustering	16
4	Method	17
4.1	Problem Formulation	17
4.2	Data Representation and Preprocessing	19
4.2.1	Scenario simulation and pulse records	20
4.2.2	Optional receiver-side degradation	20
4.2.3	Windowing and context length	20
4.2.4	Per-feature scaling	21
4.2.5	Augmentations for contrastive views	22
4.3	Transformer Encoder Architecture	22
4.4	Loss Function and Clustering Objective	25
4.5	Training Procedure	28
4.6	Evaluation Metrics	29
4.7	Baseline Methods	30
5	Experiments and Results	32
5.1	Experimental Setup	32
5.2	Datasets	32
5.2.1	Qualitative properties of the synthetic corpus	33
5.3	Hyperparameters and Training Details	33
5.4	Quantitative Results	34
5.5	Ablation Studies	34
5.6	Qualitative Analysis	35
6	Discussion	36
6.1	Interpretation of Results	36
6.2	Comparison with Related Work	36
6.3	Limitations	37
6.4	Implications	38
7	Conclusion	39
7.1	Summary	39
7.2	Answers to the Research Questions	39
7.3	Future Work	39
	References	40

A Additional Results	43
B Implementation Details	44
C Notation Reference	45

List of Figures

2.1	Illustration of the time relationship in monostatic radar: a transmission (TX) and its received echo are separated by a delay τ corresponding to twice the target range divided by the propagation speed, as in Equation (2.1).	6
2.2	Schematic of the parameters captured by a PDW for a short pulse train. Each pulse contributes a TOA (the leading edge t_n on the time axis), a pulse width (PW), a peak amplitude A_n , and a carrier RF f_n , sketched as the fast oscillation inside the middle pulse; successive pulses also define a PRI (equivalently a $\text{PRF} = 1/\text{PRI}$). The inset shows the AOA θ_n measured at the receive aperture relative to a boresight reference. A typical PDW stacks these per-pulse measurements into a vector $\mathbf{x}_n = (f_n, \text{PW}_n, t_n, A_n, \theta_n)$ that is the usual input to downstream emitter and mode reasoning [1].	7
4.1	Illustration of a PDW stream $\mathbf{x}_1, \dots, \mathbf{x}_N$ in time order. Each cell combines emitter color (fill) and operating-mode pattern; the in-cell vector index is typeset in black. The emitter legend uses color-only swatches with matching colored text; the mode legend uses grayscale pattern swatches with black text. Emitter clustering seeks a partition of indices into subsets $\mathcal{E}_k$ (constant e_n on each subset); mode clustering seeks a second partition into subsets $\mathcal{M}_\ell$ (constant m_n), while allowing mode hops within a fixed emitter.	18
4.2	Architecture of the encoder f_θ . A sequence of PDWs is first lifted to $d_{\text{model}} = 256$ by an input embedding, processed by a shared trunk of 6 transformer-encoder layers, and then split into two parallel branches with their own $256 \rightarrow 128$ projection and 2 encoder layers. Branch outputs are mapped by linear projection heads to the emitter embedding space $\mathbb{R}^8$ and the mode embedding space $\mathbb{R}^{16}$	23

List of Tables

- 5.1 Synthetic scenario corpus used for training and evaluation. Dashes mark quantities still being reconciled with the export manifest. 33
- 5.2 Primary optimisation hyperparameters for the proposed model. 34

List of Acronyms

ACC clustering accuracy

AMI adjusted mutual information

AOA angle of arrival

ARI adjusted Rand index

BERT Bidirectional Encoder Representations from Transformers

DEC deep embedded clustering

DL deep learning

ELINT electronic intelligence

ESM electronic support measures

EW electronic warfare

FFN feed-forward network

HDBSCAN Hierarchical density-based spatial clustering of applications with noise

MHA multi-head attention

ML machine learning

MLP multilayer perceptron

NMI normalized mutual information

PDW pulse descriptor word

PRF pulse repetition frequency

PRI pulse repetition interval

RADAR radio detection and ranging

RF radio frequency

TOA time of arrival

V-measure harmonic mean of homogeneity and completeness

CHAPTER 1

Introduction

1.1 Background and Motivation

Recent large-scale conflicts illustrate how contested the electromagnetic spectrum has become. Commanders rely on timely, passive sensing to build situational awareness, protect platforms, and coordinate fires without revealing their own emissions. Within EW, which aims to exploit, protect, and maneuver in the spectrum, ESM systems listen passively and supply the measurements that feed higher-level ELINT tasks such as detecting radars, grouping pulses by emitter, and inferring behavior [1, 2].

Modern radars are increasingly agile: they vary carrier frequency, pulse width, amplitude, and PRI across bursts to improve performance and survivability. That agility blurs simple fingerprints and stresses classical rule-based parsers tuned to stable pulse trains. The same physical emitter also switches *modes*—recurrent control policies that map an operational objective (for example, volume search versus track) to a characteristic regime in the pulse train. Mode awareness is therefore operationally informative: it constrains how the waveform may evolve, signals intent at a coarse level, and can help disambiguate emitters when type-level signatures overlap.

At the same time, ML and DL have become the default toolkit for sequence modeling and representation learning in other domains. Radar pulse analysis naturally presents as a stream of low-level measurements—often summarized as PDWs—where long-range dependence and mixture structure (multiple emitters superposed in time) favor learned encoders over purely hand-crafted feature pipelines, even when labels exist only for development or evaluation data.

This thesis studies whether a transformer-style encoder, trained with objectives suited to clustering, can produce embeddings in which both distinct emitters and their operating modes emerge as separable structure. The remainder of the introduction states the problem precisely, lists the research questions, and outlines scope, contributions, and ethics.

1.2 Problem Statement

Passive ESM produces a time-ordered stream of PDWs in which pulses from multiple emitters are interleaved and each emitter may change operating mode over time (Section 1.1). Dense per-pulse supervision for emitter identity and mode is rarely available outside curated datasets, yet operators still require both emitter grouping and coarse mode structure from the same measurement stream. Classical pipelines therefore often

chain deinterleaving with separate mode heuristics or supervised classifiers trained on narrow taxonomies; under agile waveforms and overlapping trains, that separation scales poorly and does not guarantee a single representation whose geometry exposes emitter- and mode-level structure simultaneously for clustering-based analysis. The problem addressed in this thesis is to learn, from such an unlabeled pulse stream, an embedding in which unsupervised clustering recovers distinct emitters and, to the extent supported by the features, operating modes, while remaining robust to mixture effects, noise, and imperfect preprocessing. Labels, when present, serve evaluation and objective specification in controlled settings rather than defining the operational learning target.

1.3 Research Questions

The thesis addresses the following research questions:

RQ1 Under practical sequence-length limits of a transformer encoder, can the model support effective deinterleaving and recover operating modes via clustering?

RQ2 Does a joint objective that couples deinterleaving with mode-aware clustering yield better clustering performance or representations than a comparable single-objective transformer baseline focused on deinterleaving alone?

1.4 Objectives and Scope

1.4.1 Objectives

The work pursues objectives that operationalize the research questions in Section 1.3. First, we specify and implement a transformer-style encoder that ingests bounded windows of PDWs from interleaved pulse streams and maps them to fixed-dimensional embeddings, together with a clustering stage that groups embeddings into emitter- and mode-level hypotheses. We then study how performance depends on practical limits on context length, mixture density, and noise, reporting agreement with reference partitions using standard scores such as NMI and ARI where ground truth is available (Chapter 5). Second, we formulate at least one joint training objective that couples sequence modeling with clustering-friendly terms, and compare it against a matched baseline trained primarily for deinterleaving or representation quality without that coupling, holding architecture capacity and data conditions as constant as feasible (Chapters 4 and 5).

1.4.2 Delimitations

The scope is deliberately narrow. We operate at the level of PDWs or equivalent tabular pulse features supplied to the model; pulse detection, RF conditioning, antenna-level

processing, and geometry-driven fusion across platforms are out of scope. The focus is offline or batch analysis of recorded streams rather than hard real-time implementation on embedded hardware, and we do not address full operational emitter naming against classified libraries. That choice excludes an *online* regime in which models update continuously from a live pulse stream under latency and memory budgets: training and evaluation here assume access to bounded windows that can be revisited, shuffled, and mini-batched, which sidesteps causal filtering, drift handling, and resource accounting that a streaming deployment would require. Consequently, conclusions concern representation quality and clustering agreement on curated windows rather than end-to-end tracking performance over arbitrarily long horizons. When labels exist, they serve evaluation, ablations, and controlled supervision experiments, not the definition of the fielded learning target stated in Section 1.2. Finally, we do not claim field validation on classified measurements; conclusions are drawn from synthetic or curated datasets together with stress tests that mimic deinterleaving noise (Chapter 5).

1.5 Contributions

The main contributions of this thesis are:

- Placeholder contribution.
- Placeholder contribution.

1.6 Ethical and Societal Considerations

Methods for passive ESM and ELINT sit squarely in dual-use territory: the same representations that support defensive situational awareness, spectrum management, and safety-of-navigation applications can, in principle, support targeting or surveillance against specific platforms if deployed without appropriate policy, oversight, and legal constraints. This thesis develops ML techniques on abstract pulse-level features and does not address deployment, rules of engagement, or classification of live operational data; nevertheless, the capability to group emitters and modes from unlabeled streams should be understood as a general-purpose building block whose societal consequences depend on the institutional context in which it is used.

From a data-ethics perspective, the experimental work relies on controlled or synthetic PDW streams rather than collections derived from live emitters, which limits privacy and classification sensitivity concerns at the cost of weaker guarantees about behavior under classified or vendor-specific waveforms. When such methods are extended toward operational data, stewards must enforce access control, purpose limitation, and retention policies consistent with national regulation and organizational policy, and they should document provenance and limitations so that operators do not over-trust cluster labels that lack ground truth.

Training modern DL encoders incurs non-negligible energy use for computation and cooling; relative to lightweight classical deinterleaving heuristics, transformer-scale models increase the carbon footprint of experimentation and of any future always-on training loops. Mitigation follows standard practice: reuse checkpoints, avoid redundant hyperparameter sweeps, prefer smaller models when they meet accuracy targets, and report approximate training cost where it informs reproducibility.

At a societal level, automation of emitter and mode analysis may concentrate analytic capacity in well-resourced actors, widen asymmetries in electromagnetic situational awareness, and reduce the visibility of human expert judgment if clusters are treated as authoritative without audit trails. Responsible research therefore favors transparent evaluation protocols, clear statements of failure modes under noise and mixture, and conservative claims about operational readiness.

1.7 Thesis Outline

The remainder of the thesis is organized as follows. Chapter 2 supplies textbook-style background on pulsed radar, PDWs, emitters and operating modes, the ELINT processing chain, and the ML and transformer concepts needed to read the method. Chapter 3 turns to the literature, contrasting classical emitter identification with deep supervised radar models, deep clustering objectives, sequence transformers, and work on joint or hierarchical cluster structure; each subsection closes with the limitation that motivates the present approach. Chapter 4 states the formal problem, documents data representation and preprocessing (Section 4.2), specifies the dual-branch encoder and losses (Sections 4.3 and 4.4), describes training and metrics, and lists baselines. Chapter 5 reports hardware and software setup, the synthetic scenario corpus (Section 5.2), hyperparameters, quantitative scores, ablations, and qualitative analyses. Chapter 6 interprets findings, positions them against prior art, states limitations—including simulator-only evidence and operational gaps—and discusses implications. Chapter 7 summarizes answers to the research questions and outlines future work.

CHAPTER 2

Background

2.1 Radar Systems and Passive Reception

2.1.1 Electromagnetic Pulses for navigation, surveillance, and control

Engineered RADAR grew from wartime range finding into a family of microwave systems used for air traffic control, weather mapping, altimetry, ship and vehicle navigation, target tracking, and spectrum monitoring next to commercial communication services [2]. The common thread is practical: launch a controlled electromagnetic disturbance, let it scatter, and read geometry and motion from the returned field. Fundamentally, electromagnetic radiation is produced whenever electric charges accelerate, such as when alternating current flows in an antenna. This disturbance travels outward as a wave at a predictable speed—the speed of light in air. For radar applications, each transmission can be understood as a short, localized wave packet. By carefully measuring how long it takes for this packet to return after reflecting off an object, and knowing the propagation speed, the distance to the object can be calculated directly [2].

2.1.2 Time-of-Flight Ranging and Ambiguity Fundamentals

At its core, radar sensing uses the physics of wave propagation to measure distance. A transmitter emits a brief electromagnetic pulse, which travels at speed c (the speed of light in air), reflects from an object, and returns as an echo. By measuring the elapsed time τ between emission and reception—known as the round-trip delay—the system can infer the distance to the object.

In the canonical case, a single (monostatic) radar platform handles both transmission and reception, with the wave traversing to the target and back. The fundamental relation connecting time delay to range follows directly from the definition of speed:

$$R = \frac{c\tau}{2}, \tag{2.1}$$

where R is the target range and c is the wave speed [2]. The factor of 2 accounts for the outbound and return paths.

A similar approach is found in animal echolocation: for instance, a bat produces a short sound pulse and estimates the distance to an obstacle or prey by interpreting the delay before it hears the echo, substituting sound waves for electromagnetic ones [2].

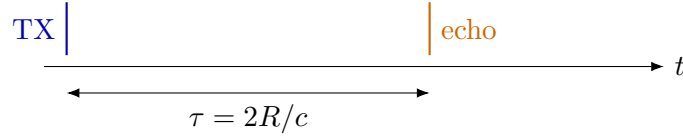


Figure 2.1. Illustration of the time relationship in monostatic radar: a transmission (TX) and its received echo are separated by a delay τ corresponding to twice the target range divided by the propagation speed, as in Equation (2.1).

Practical radar systems emit periodic pulses, each separated by a pulse repetition interval (PRI) or, equivalently, a pulse repetition frequency (PRF). A core challenge appears when echoes from earlier pulses arrive after a new pulse is sent. This leads to *range ambiguity*, where it is unclear which transmitted pulse a received echo should be attributed to. The largest range without ambiguity—the *unambiguous range*—is set by the time between pulses:

$$R_u = \frac{c}{2 \text{PRF}} = \frac{c \text{PRI}}{2}. \quad (2.2)$$

Signal processing techniques such as matched filtering and coherent integration increase detection performance within this range, but no algorithm can resolve targets unambiguously beyond R_u as limited by the physics of pulse timing [2].

2.1.3 Transmitting radars versus receive-only systems

A pulsed radar platform both illuminates and listens, so it can exploit a known transmit waveform when forming correlations and Doppler filters [2]. Passive ESM and ELINT systems instead sit on the sidelobe or distant mainlobe energy of someone else’s transmitter. They do not control P_t , antenna gain schedules, or internal timing laws, so cooperative echo models are largely unavailable. Inference therefore leans on measured per-pulse parameters (carrier, arrival time, width, angle, amplitude) rather than on matched filtering against a secret template. That posture is exactly what motivates the PDW abstraction developed below and detailed further in Section 2.2.

2.1.4 From passband samples to a list of pulses

A typical front end produces a digitized passband or near-baseband record $x[n]$ at a sample rate high enough to capture the emitted bandwidth. Energy detection marks candidate pulse intervals against thermal noise, after which parameter estimation attaches scalars to each event. Spectral location of narrowband energy is naturally read from a windowed discrete Fourier transform,

$$X_m[k] = \sum_{n=0}^{N-1} w[n] x[n + mH] e^{-i2\pi kn/N}, \quad (2.3)$$

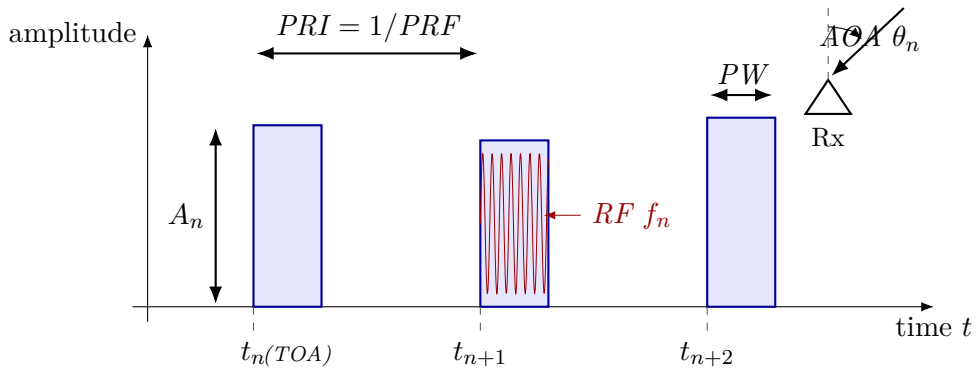


Figure 2.2. Schematic of the parameters captured by a PDW for a short pulse train. Each pulse contributes a TOA (the leading edge t_n on the time axis), a pulse width (PW), a peak amplitude A_n , and a carrier RF f_n , sketched as the fast oscillation inside the middle pulse; successive pulses also define a PRI (equivalently a $\text{PRF} = 1/\text{PRI}$). The inset shows the AOA θ_n measured at the receive aperture relative to a boresight reference. A typical PDW stacks these per-pulse measurements into a vector $\mathbf{x}_n = (f_n, \text{PW}_n, t_n, A_n, \theta_n)$ that is the usual input to downstream emitter and mode reasoning [1].

with window $w[n]$, hop H , and bin index k tied to physical frequency through the tuning plan [2]. Time of arrival and pulse width are then obtained from correlation against a template or from threshold crossings on the envelope, depending on the receiver [1]. The intermediate waveform is discarded once those numbers are stable, which is why later learning stages never see the raw $x[n]$.

2.1.5 Pulse descriptor words and the pulse train

After detection and parameter estimation in the pipeline of Section 2.2, each accepted pulse is summarized by a *PDW*: a fixed-layout record of measured pulse parameters that discards intermediate waveforms while retaining the quantities downstream algorithms consume [1]. Index pulses in time order by $n \in \{1, 2, \dots\}$ and write the n th descriptor as

$$\mathbf{x}_n = (f_n, \text{PW}_n, t_n, A_n, \theta_n, \dots)^\top \in \mathbb{R}^d, \quad (2.4)$$

where f_n is the measured carrier frequency, PW_n the pulse width, t_n the TOA of a chosen reference point (often the leading edge), A_n a scalar amplitude or energy proxy, and θ_n the AOA when direction finding exists. The ellipsis stands for any extra columns the sensor exports (sub-band tags, polarization flags, intra-pulse modulation labels); d is fixed once the column order is frozen. Figure 2.2 matches the minimal five-field template implied by Equation (2.4).

For a single coherent pulse train, successive TOAs define inter-pulse intervals

$$\tau_n = t_n - t_{n-1}, \quad n \geq 2, \quad (2.5)$$

so a nominal PRI is read from τ_n and a PRF follows as $\text{PRF}_n = 1/\tau_n$ when the train is

close to periodic. Agile PRI schedules replace one constant τ_n with structured sequences; either τ_n is stored per pulse or extra columns carry stagger metadata, but each index n still labels one row in $\mathbb{R}^d$ as in Equation (2.4).

Measured descriptors obey $\mathbf{x}_n = \mathbf{g}(\mathbf{s}_n) + \boldsymbol{\epsilon}_n$ with latent physical state $\mathbf{s}_n$, deterministic sensor map $\mathbf{g}$, and aggregate error $\boldsymbol{\epsilon}_n$ from noise, quantization, and estimator bias. Weak pulses inflate effective noise or leave fields blank; false alarms inject samples with no meaningful $\mathbf{s}_n$. *Deinterleaving* (Section 2.2) partitions detections into trains so that Equation (2.5) is interpretable inside each train; mixture, misses, and spurious hits then show up as structured outliers in the sequence $(\mathbf{x}_n)$.

In later chapters, the model input is a *time-ordered PDW sequence* $(\mathbf{x}_1, \dots, \mathbf{x}_T)$, that is, a *pulse train* (possibly after association and sorting) rather than the raw digitized waveform. Each $\mathbf{x}_n$ is one token in a length- T sequence whose coordinates carry explicit physical units and semantics, which matters when designing encoders and augmentation.

2.1.6 Physical sources and transmit schedules

In passive ESM and ELINT, a *radar emitter* means a physical transmitter whose pulses line up in time and frequency strongly enough to be treated as one source [1]. An *operating mode* is the instantaneous transmit recipe: carrier plan, pulse width, repetition pattern, beam motion, and any programmed switching rules among them [1, 2]. The same hardware cycles among modes as tasks change, so a timeline of PDWs mixes mode segments even when the physical box is fixed. Different platforms can also implement the same functional mode with different numerical parameters, for example two trackers with distinct nominal PRFs [1].

Illustrative mode families include wide-area search, acquisition and track, fire-control illumination, weather sensing, and terminal guidance support [2]. Those roles push different combinations of PRI, dwell, agility, and scan law, which is why operators care both about identity and about what the transmitter is doing at a given time [1, 2].

The relationship between emitters and modes is many-to-many: one emitter runs many modes over a mission, and different emitters can realize similar modes with different parameter sets [1]. The rest of this chapter walks through the passive processing chain (Section 2.2) and then supplies the learning background used in the method.

2.2 Signal Processing Pipeline for Passive ELINT

In EW and ESM applications, measured radar energy passes through a staged *signal processing pipeline* before it reaches higher-level ELINT reasoning. The exact implementation varies with platform, band, and mission software, but the logical chain is broadly stable across textbooks and system descriptions.

2.2.1 Front-end reception, detection, and pulse-parameter estimation

At the *front end*, antennas and RF conditioning capture emissions over one or more instantaneous bandwidths, often after wideband digitization or channelized sub-band processing. A *detection* stage then marks pulse-like events against a noise and clutter background, producing a sparse list of candidate pulse times (and sometimes coarse frequency hints).

Given detections, *parameter estimation* attaches per-pulse measurements such as carrier frequency, pulse width, time of arrival, direction of arrival when available, and amplitude or signal-to-noise proxies. These estimates are noisy, can be missing for weak emitters, and may include false alarms that are not associated with any physical radar.

2.2.2 Deinterleaving and pulse-train formation

Sorting and *deinterleaving* group pulses into coherent pulse trains by exploiting periodic structure in inter-pulse timing and consistent parameter tracks across time. The output of this stage is commonly packaged as a stream of PDWs (or closely related pulse descriptor records), which form the usual interface to downstream tasks such as emitter identification and mode analysis (Section 2.1.5).

Operationally, deinterleaving developed from deterministic temporal rules toward multi-parameter clustering and, more recently, toward learned sequence models [3]. In comparatively sparse environments with simple pulse trains, PRI inferred from TOA differences often behaved like a stable timing signature, so early algorithms searched for repetition by accumulating histograms of inter-pulse intervals. Cumulative and sequential difference histograms (CDIF and SDIF) are representative: they “vote” for candidate PRI values through peaks in the difference domain and remain standard pedagogical references [1, 3]. Their accuracy rests on approximately regular timing; dense mixing, harmonic ambiguity, false peaks from cross-emitter differences, and missing pulses that break sequential differencing all erode reliability [3].

Once PRI agility became widespread, many systems augmented timing with unsupervised structure in joint spaces of RF, pulse width, AOA, and related measurements, using methods such as k -means or density-based clustering [3, 4]. That shift treats deinterleaving as geometric separability in hand-crafted feature space, but performance still depends on scaling, metric choices, and hyperparameters (for example cluster counts or neighborhood radii) that are difficult to set when emitters are unknown a priori [3].

Modern emitters compound the difficulty through overlapping pulse trains, rapid mode changes, and low-observable emissions that increase loss and measurement jitter [3]. The PDW representation in Section 2.1.5 and the learning-based tools in later sections of this chapter target reasoning under those conditions.

In classical ELINT architectures, specifications therefore treat train formation as an

early, largely discrete step: deinterleaving is completed first so that downstream logic receives one dominant hypothesis per physical emission path, or a short ranked list, rather than a time-interleaved mixture [1].

2.2.3 Emitter reporting and equipment identification

Emitter reports are assembled after train formation, summarizing each train with aggregated PDW statistics, tentative equipment identities from lookup tables, and temporal extent. Library-driven identification and threat correlation consume those reports together with operator context.

2.2.4 Operating-mode analysis and waveform libraries

Mode classification is then commonly framed as matching the train against a *mode library* of nominal RF bands, PRI/PRF families, stagger laws, and scan programs, so that declared modes remain anchored to catalogued waveform templates. When labels are incomplete, unsupervised or weakly supervised grouping over pulse trains plays the same operational role as *mode clustering* at the library boundary: it proposes coherent behavior segments that analysts later align to doctrine or intelligence products.

2.2.5 Emitter geolocation, fusion, and downstream displays

Positional estimation—for example AOA intersections from direction-finding networks, time-difference-of-arrival fixes along baselines, or fused tracks across collectors—consumes the same deinterleaved associations, which isolates geometry and timing errors from the sorting stage. Later stages may maintain libraries, threat lists, and operator displays, but for the learning problem in this thesis the critical contract is that the model consumes the deinterleaved PDW stream produced here.

2.3 Machine Learning Fundamentals

ML problems are often categorized by the supervision available at training time. In the supervised setting, a learner fits a mapping from inputs to labels by minimizing an empirical risk over a finite set of annotated examples. Unsupervised learning instead seeks structure in the input distribution itself, for example through density estimation, dimensionality reduction, or clustering, without externally supplied targets. Self-supervised learning occupies an intermediate regime: targets are derived automatically from the data through a designed pretext (for example predicting masked parts of an input or contrasting augmented views), which supplies learning signal without manual annotation. Across these regimes, a common formalism is empirical risk minimization, in which one minimizes an average training loss with respect to model parameters and relies on optimization, regularization, and held-out evaluation to control generalization.

Much of modern DL practice is best understood as *representation learning*: a parametric encoder maps high-dimensional observations into a lower-dimensional vector space whose geometry is shaped by the training objective. Downstream tasks—classification, clustering, retrieval, or sequence prediction—then operate on these *embeddings* rather than on raw sensor coordinates. The thesis later instantiates this pattern for PDW streams by training an encoder whose outputs are intended to expose emitter and mode structure (Chapter 4).

LeCun [5] argues for organizing this picture around *joint embeddings*. In joint embedding predictive architectures, two related observations x and y (for example two temporal crops of the same scene) are mapped to representations s_x and s_y by encoders, and a predictor proposes $\tilde{s}_y$ from s_x (possibly with latent variables that capture multi-valued futures). Compatibility is scored by an energy $D(s_y, \tilde{s}_y)$ that measures prediction error in representation space rather than in pixel space or other raw domains. As LeCun [5] puts it,

“The energy is simply the prediction error in representation space.”

The same source emphasizes that the main advantage of this design is that prediction happens after irrelevant variability has been discarded by the encoders, so that the learning problem is not to reconstruct every detail of y , but to align abstract embeddings that already summarize what must stay predictable.

When D is instantiated as a squared Euclidean distance between embeddings, the landscape of D with respect to s_y is piecewise quadratic under standard predictor parameterizations. LeCun [5] uses this viewpoint to explain why, in discrete-latent variants, low-energy regions can be visualized as competing wells in embedding space:

“In representation space, the energy will be the minimum of K quadratic energy wells.”

Colloquially, one can read this as the claim that *once* suitable embeddings and predictors have been identified, much of the remaining compatibility training reduces to a family of quadratic matching problems in representation coordinates; the scientific effort then concentrates on representation geometry, collapse prevention, and task-relevant invariances rather than on inventing elaborate scalar losses on raw inputs. The clustering objectives used in Chapter 4 should be interpreted against that backdrop: they reshape embedding space, while the parametric form of the pointwise loss can remain simple.

2.4 Clustering Methods

Placeholder paragraph.

2.5 Deep Representation Learning

Many DL pipelines replace hand-crafted features with a parametric map that is trained end-to-end from data. The central object is an *encoder* $f_{\theta}: \mathcal{X} \rightarrow \mathbb{R}^d$, implemented in practice as a stack of nonlinear layers (for example, convolutions over a grid, recurrence over time, or attention over a sequence) followed by a pooling or readout head that yields a fixed-dimensional embedding $\mathbf{z} = f_{\theta}(\mathbf{x})$. The design question is which training signal makes $\mathbf{z}$ useful for the downstream task when labels are scarce or absent.

In supervised learning, f_{θ} is trained together with a classifier so that $\mathbf{z}$ separates classes under a cross-entropy or margin loss. When cluster structure is the target but assignments are unknown, the same encoder can instead be trained with objectives that encourage smoothness, invariance to nuisance transformations, or agreement with a clustering hypothesis in embedding space. *Self-supervised* learning occupies an intermediate regime: the model predicts surrogate targets constructed from the input itself (for example, masked tokens, relative positions, or transformed views), so that f_{θ} learns semantics without human labels [6].

Contrastive objectives implement instance discrimination by treating two augmentations of the same example as a positive pair and other examples in a minibatch (or a memory bank) as negatives. Let $\mathbf{z}_i$ and $\mathbf{z}_i^+$ denote normalized embeddings of two views of sample i , let $\text{sim}(\cdot, \cdot)$ denote cosine similarity, and let $\tau > 0$ be a temperature. A standard *InfoNCE*-style loss takes the form

$$\mathcal{L}_{\text{NCE}}(i) = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_i^+)/\tau)}{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_i^+)/\tau) + \sum_{j \neq i} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j^+)/\tau)}, \quad (2.6)$$

which lower-bounds mutual information between representations of different views under suitable generative assumptions [7]. Large-scale implementations such as SimCLR combine strong augmentations with wide embeddings and many negatives drawn from the same minibatch [6]. Equation (2.6) should be read as a template: the positive set can be enlarged, negatives can be drawn from a queue, and the similarity can be implemented with a learned projection head without changing the underlying contrastive principle.

Pretext tasks offer an alternative route: the network solves an auxiliary problem (predicting rotations, solving jigsaw permutations, inpainting masked regions) whose solution is incidental, but the representation $\mathbf{z}$ becomes predictive of semantically stable factors. Such objectives can be combined with clustering when the end goal is partition discovery rather than linear probe accuracy on a fixed label set. Section 3.3 surveys deep embedded clustering and prototype-based alternatives that couple encoder training with discrete structure, together with the gap relative to nested emitter–mode organization in ELINT streams.

Sequence encoders based on self-attention, including the transformer blocks reviewed

in Section 2.6, instantiate f_θ when $\mathbf{x}$ is an ordered set of tokens (for example, a window of PDWs). The Method chapter builds on this stack: a contrastive or clustering-friendly loss shapes the embedding space, while the attention-based encoder models temporal context within each window.

2.6 The Transformer Encoder

The transformer architecture replaces recurrence with a stack of layers built around *self-attention*, so that every position in a sequence can directly attend to every other position in parallel [8]. Encoder-only stacks, as in BERT [9], map an input sequence to contextualised token representations of the same length and are a natural choice when the downstream task is representation learning or clustering over tokens rather than autoregressive generation.

Scaled dot-product attention. Attention maps three matrices of *queries* $\mathbf{Q}$, *keys* $\mathbf{K}$, and *values* $\mathbf{V}$ to an output matrix of the same leading dimension as $\mathbf{Q}$. Each row of $\mathbf{Q}$ scores all rows of $\mathbf{K}$ through inner products; the scores are scaled, normalised with a row-wise softmax, and used to form a convex combination of the rows of $\mathbf{V}$. With d_k the dimension of each query and key vector (column width of $\mathbf{Q}$ and $\mathbf{K}$ after the usual transpose convention), the mapping is

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (2.7)$$

The scaling by $\sqrt{d_k}$ keeps the inner products from growing too large in magnitude as d_k increases, which would otherwise drive the softmax into near one-hot rows and yield vanishing gradients [8]. Equation (2.7) is reused later as the core attention primitive inside the encoder.

Self-attention and MHA. In *self-attention*, $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are all linear projections of the same input sequence layout, so a token’s representation is updated by aggregating information from the full sequence. MHA runs h attention mechanisms in parallel on different learned subspaces, concatenates the head outputs, and applies one more linear map to restore the model width [8]. The heads specialise to different relational patterns while sharing the same residual and normalisation wrapper as a single conceptual operator.

Positional information. Self-attention is permutation-equivariant in its inputs: reordering the rows of $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ in the same way reorders the output rows accordingly, but does not otherwise encode order. Transformer encoders therefore add a positional signal to token embeddings before the first layer, for example fixed sinusoidal features or learned position embeddings added element-wise to each token vector [8,

9]. When the raw inputs already carry absolute or relative timing, as is common for irregularly sampled sequences, practitioners sometimes omit a separate positional encoding and rely on those measurements instead; the method chapter returns to this choice for PDW windows.

Encoder layer. A standard transformer *encoder layer* alternates two sub-layers: MHA followed by a position-wise FFN applied independently at every position. Each sub-layer uses a residual connection and layer normalisation [8]. The FFN is typically a two-layer MLP with a wide inner dimension and a nonlinear activation, acting as the per-token nonlinearity while MHA handles cross-token mixing. Stacking N such layers yields a deep encoder that maps an input sequence $(\mathbf{x}_1, \dots, \mathbf{x}_L)$ to contextualised representations $(\mathbf{h}_1^{(N)}, \dots, \mathbf{h}_L^{(N)})$, which downstream heads can pool or read out for sequence-level or token-level predictions.

Computational cost. In the usual dense implementation, each self-attention block materialises an $L \times L$ matrix of token–token scores before the softmax, so time and activation memory for that block scale as $\mathcal{O}(L^2)$ in the sequence length L at fixed embedding width, up to constants from the number of heads and projection ranks [8]. Repeating the block across depth multiplies that cost by the number of layers, which motivates hard caps on L , sparse or low-rank attention variants, or chunked processing when inputs are long; the method chapter states the windowing choice adopted here.

CHAPTER 3

Related Work

3.1 Classical Radar Emitter Identification

Placeholder paragraph.

3.2 Deep Learning for Radar Signal Analysis

Placeholder paragraph.

3.3 Deep Clustering and Joint Representation Learning

Classical clustering assumes that either raw features or a fixed embedding already reveal the desired partition. DEC departs from that split by learning an encoder jointly with a soft assignment to a small set of trainable cluster centroids in embedding space [10]. Training alternates between computing soft assignments from the current encoder and refining the encoder with a Kullback–Leibler objective that sharpens confident assignments toward an auxiliary target distribution, so that the network discovers both features and cluster structure. Follow-on work tightens initialization, adds reconstruction constraints, or refines the target distribution; the common theme is that discrete structure is not fixed in advance but co-adapts with the representation.

SwAV and related *prototype* methods replace dense pairwise contrastive negatives with a prediction problem onto a modest codebook of cluster prototypes, swapping predicted codes between augmented views to avoid representation collapse while still encouraging invariance [11]. They sit between purely self-supervised contrastive pre-training [6] and hard assignment k -means: gradients still flow through discrete-like targets, but the geometry is anchored by learned prototypes rather than by instance indices alone.

Several lines extend deep clustering to *multiple* partitions or hierarchical labels, for example by stacking separate assignment heads, alternating optimization over two clusterings, or enforcing orthogonality constraints between subspaces. The gap relevant to this thesis is narrower: emitter identity and operating mode are not arbitrary independent clusterings—modes nest inside emitters—yet many published pipelines still treat a single flat label space or a purely supervised multi-head classifier. Work that jointly addresses deinterleaving-style separation and finer-grained mode structure

from *unlabeled* PDW windows with a shared transformer encoder remains comparatively sparse, which motivates the method in Chapter 4.

3.4 Transformers for Time Series and Sets

Placeholder paragraph.

3.5 Joint and Multi-Task Clustering

Placeholder paragraph.

CHAPTER 4

Method

4.1 Problem Formulation

This chapter studies unsupervised structure recovery from a time-ordered stream of PDWs produced by an ESM or deinterleaving front end, as outlined in Section 2.2. The goal is twofold: associate each pulse with a physical *emitter instance* and, within that instance, with an *operating mode*, without prior knowledge of how many emitters or modes appear in the data. Both quantities are latent because labels are scarce in ELINT settings and because novel emitters and modes must be accommodated.

Observed input

Let a sequence of N pulses be represented by feature vectors

$$\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N), \quad \mathbf{x}_n \in \mathbb{R}^d, \quad (4.1)$$

where each $\mathbf{x}_n$ encodes the measured pulse parameters associated with one PDW (carrier frequency, pulse width, timing, spatial cues, and any auxiliary fields). The sequence may contain irregular sampling, missing pulses, and spurious detections inherited from upstream processing; robustness to such effects is a design constraint rather than part of the formal specification.

Latent emitter and mode assignments

For each index $n \in \{1, \dots, N\}$, let e_n denote the emitter identity responsible for pulse n and let m_n denote the operating mode within that emitter. These variables take values in finite label sets whose cardinalities

$$K_{\text{em}} = |\{e_1, \dots, e_N\}|, \quad K_{\text{md}} = |\{m_1, \dots, m_N\}| \quad (4.2)$$

are *unknown a priori* and need not match the number of physical platforms in the environment (for example, co-located antennas may map to a single inferred emitter index, or a single platform may split across indices when observations are ambiguous). The pair (e_n, m_n) is not observed at training time; the learning problem is to infer representations from which both clusterings can be recovered reliably.

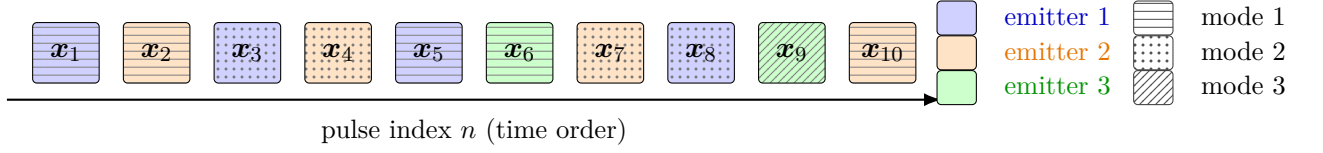


Figure 4.1. Illustration of a PDW stream $\mathbf{x}_1, \dots, \mathbf{x}_N$ in time order. Each cell combines emitter color (fill) and operating-mode pattern; the in-cell vector index is typeset in black. The emitter legend uses color-only swatches with matching colored text; the mode legend uses grayscale pattern swatches with black text. Emitter clustering seeks a partition of indices into subsets $\mathcal{E}_k$ (constant e_n on each subset); mode clustering seeks a second partition into subsets $\mathcal{M}_\ell$ (constant m_n), while allowing mode hops within a fixed emitter.

Induced partitions, PDW streams, and mode hopping

Figure 4.1 sketches a short PDW stream in chronological order. Fill color encodes the latent emitter index e_n ; pulses that share a color belong to the same emitter-specific subset of indices even though they need not be contiguous in time (interleaved emissions from several RADAR platforms are typical). Each stream cell uses that emitter fill *and* a grayscale pattern for operating mode m_n ; the vector label $\mathbf{x}_i$ inside each cell is drawn in black for legibility. The right-hand legends separate the two cues: emitter entries are color-only (no pattern), and mode entries are monochrome pattern tiles with black text. The formal partitions and mode hopping are stated mathematically in the text below.

Emitter clustering is the recovery of the partition of $\{1, \dots, N\}$ into emitter-coherent index sets

$$\mathcal{E}_k = \{n \in \{1, \dots, N\} : e_n = k\}, \quad k \in \{1, \dots, K_{\text{em}}\}, \quad (4.3)$$

so that $\mathbf{x}_n$ and $\mathbf{x}_{n'}$ are grouped together if and only if $e_n = e_{n'}$. Equivalently, each $\mathcal{E}_k$ is the preimage of label k under the map $n \mapsto e_n$, and $\{\mathcal{E}_1, \dots, \mathcal{E}_{K_{\text{em}}}\}$ is pairwise disjoint with union $\{1, \dots, N\}$.

Mode clustering targets an analogous partition into mode-coherent subsets,

$$\mathcal{M}_\ell = \{n \in \{1, \dots, N\} : m_n = \ell\}, \quad \ell \in \{1, \dots, K_{\text{md}}\}, \quad (4.4)$$

which groups pulses that share the same operating mode regardless of which physical emitter produced them. The two partitions are coupled because (e_n, m_n) is generated jointly: restricting to a fixed emitter k , the induced set of mode labels

$$\mathcal{L}_k = \{m_n : n \in \mathcal{E}_k\} \quad (4.5)$$

has cardinality $L_k = |\mathcal{L}_k|$ bounded by the number of distinct waveform programmes that emitter k exercises. In many RADAR systems, mode hopping switches among a small, essentially finite catalogue of modes, so L_k is modest—often on the order of single-digit integers—even though the temporal pattern $(m_n)_{n \in \mathcal{E}_k}$ is not constant. The learning problem is to infer partitions that match $\{\mathcal{E}_k\}$ and $\{\mathcal{M}_\ell\}$ up to independent relabellings (permutations of emitter symbols k and mode symbols ℓ) from $\mathbf{X}$ alone,

using the dual embeddings in Equation (4.6).

Desired outputs: dual embeddings and discrete labels

The parametric model considered in this thesis maps each input sequence to two parallel trajectories in Euclidean embedding spaces,

$$\mathbf{Z}^{\text{em}} = (\mathbf{z}_1^{\text{em}}, \dots, \mathbf{z}_N^{\text{em}}), \quad \mathbf{z}_n^{\text{em}} \in \mathbb{R}^p, \quad \mathbf{Z}^{\text{md}} = (\mathbf{z}_1^{\text{md}}, \dots, \mathbf{z}_N^{\text{md}}), \quad \mathbf{z}_n^{\text{md}} \in \mathbb{R}^q, \quad (4.6)$$

with dimensions p and q fixed by the architecture. Discrete emitter labels $\hat{e}_n$ and mode labels $\hat{m}_n$ are obtained *post hoc* by clustering $\{\mathbf{z}_n^{\text{em}}\}_{n=1}^N$ and $\{\mathbf{z}_n^{\text{md}}\}_{n=1}^N$, or by reading off prototype assignments when the training objective includes cluster parameters. Thus the immediate trainable targets are the continuous vectors in Equation (4.6); hard partitions are induced indirectly.

The two trajectories serve different roles. Emitter-oriented vectors $\mathbf{z}_n^{\text{em}}$ are primarily a vehicle for estimating consistent emitter membership; their main deliverable is a per-pulse emitter index suitable for downstream ELINT workflows such as emitter counting and localisation. Mode-oriented vectors $\mathbf{z}_n^{\text{md}}$ additionally aim to summarise mode-dependent waveform behaviour so that operating modes can be inspected, compared, or matched against a library of known modes once such references become available.

Architecturally, both streams are produced from a shared encoder f_{θ} with task-specific heads (see Section 4.3); in deployment, emitter embeddings may be less critical to retain after discrete $\hat{e}_n$ have been fixed, whereas mode embeddings (or intermediate encoder activations feeding the mode head) are natural inputs to specialised mode classifiers.

Learning objective

The optimisation problem couples representation learning with clustering-friendly losses so that the geometry of $\mathbf{Z}^{\text{em}}$ and $\mathbf{Z}^{\text{md}}$ aligns with emitter and mode structure, respectively. The concrete loss terms and training protocol are given in Sections 4.4 and 4.5.

4.2 Data Representation and Preprocessing

ELINT data are difficult to obtain at scale, and operational recordings are often sensitive. This work therefore relies on synthetic pulse trains from a proprietary scenario simulator at Saab, which yields controlled scenarios and full supervision of emitter identity and operating mode for evaluation, while the learning objectives remain unsupervised with respect to those labels. Earlier KTH degree projects hosted at Saab document synthetic radar-ML pipelines at a comparable level of detail, including scenario-driven pulse generation and explicit noise models [12, 13].

4.2.1 Scenario simulation and pulse records

The simulator produces *scenarios*: time-ordered pulse-level timelines for one or more notional emitters and a receiving platform. Each pulse is described by the continuous fields that enter the PDW representation used here (including TOA, RF, pulse width, and amplitude on a logarithmic scale), together with discrete labels for emitter identity and operating mode that exist only for evaluation. Jonsson [12] describes the unclassified OpenModesty tool in the same lineage: flight paths and emitter placements define a geometry, pulses are emitted and intercepted one at a time, and the simulator can apply modulations and controlled perturbations consistent with notional radar behavior. The present implementation is proprietary; the important methodological point is that every pulse in the export carries consistent timing and parameter fields so that windows formed below correspond to reproducible segments of a simulated electromagnetic timeline.

Scenarios differ in the number of emitters, dwell in each mode, and simulated horizon so that training covers diverse traffic densities; exact counts and split assignments are deferred to Chapter 5.

4.2.2 Optional receiver-side degradation

Even when emitter physics are ideal in the simulator, downstream ESM processing introduces structured errors. Jonsson [12] distinguishes *dropped pulses*, where a transmitted pulse is not registered by the receiver chain, from *spurious pulses*, where extra pulses appear in a track because deinterleaving or detection stages attach energy to the wrong pulse train. Svensson [13] likewise emphasizes controlled drop, spurious, and additive Gaussian regimes when studying PRI sequences. When the experimental programme applies such degradations, they are implemented as stochastic edits on the ordered pulse list *before* windowing: each pulse may be removed with a scenario- or split-specific drop probability, and spurious insertions may be drawn by copying and perturbing existing pulses or by drawing alternative parameter vectors from a bounded rule set. Discrete grids of drop and spurious rates, rather than only single operating points, simplify reporting across noise sweeps in Chapter 5.

Algorithm 1 is only a bookkeeping template: concrete probabilities, conditioning on track boundaries, and whether noise is re-drawn per window or fixed per scenario appear in the experiment log together with the simulator export manifest.

4.2.3 Windowing and context length

The encoder operates on fixed-length windows because standard self-attention builds an $L \times L$ compatibility matrix between tokens at each layer, so time and memory scale as $\mathcal{O}(L^2)$ in the window length L at fixed model width, repeated across depth. Let L denote the window length (in pulses) fed to the model; experiments use $L = 1024$. Each

Algorithm 1 Optional stochastic degradation of an ordered pulse list (conceptual).

Require: Ordered pulses $\{p_1, \dots, p_N\}$, drop probability $\rho_{\text{drop}} \in [0, 1)$, spurious probability $\rho_{\text{spu}} \in [0, 1)$

Ensure: Degraded list of pulses preserving time order where applicable

```

1:  $\mathcal{L} \leftarrow \text{copy}(\{p_1, \dots, p_N\})$ 
2: for each pulse  $q$  in  $\mathcal{L}$  scanned in forward time order do
3:   if uniform draw  $u < \rho_{\text{drop}}$  then
4:     remove  $q$  from  $\mathcal{L}$ 
5:   end if
6: end for
7: for each remaining pulse  $q$  in  $\mathcal{L}$  do
8:   if uniform draw  $u' < \rho_{\text{spu}}$  then
9:     insert a spurious pulse after  $q$  using the chosen perturbation rule (duplicate
       with noise or independent draw)
10:  end if
11: end for

```

scenario is partitioned into contiguous windows of length L . If the remainder after the last full window is shorter than L , it is dropped in the present pipeline; padding with an attention mask would retain the tail without changing the architecture and is left as a straightforward extension.

Training windows are extracted with a stride of $L/2$, so consecutive windows overlap by half their length. This increases the number of training pulses seen per scenario and reduces sensitivity to the particular alignment of window boundaries relative to mode transitions. Validation and test windows use stride L (no overlap), so that metrics are not inflated by near-duplicate windows.

Interleaved traffic and heterogeneous dwell times imply that pulse counts per emitter and per operating mode vary widely inside a window, and the effective PRI structure can make the number of pulses per emitter differ across the corpus even when scenario durations match [12]. Some windows therefore contain few or no pulses from a rare emitter or a short-lived mode, even when that label is frequent in the parent scenario, and dropped tails can remove trailing segments entirely. These effects induce a form of *label incidence imbalance* at the window level that is distinct from corpus-level class histograms and should be read alongside the reporting conventions in Chapter 5.

4.2.4 Per-feature scaling

Amplitude is already provided on a logarithmic scale (decibels). Continuous fields are transformed before windowing so that statistics are computed on the training split only and then applied to validation and test data. The log-amplitude, carrier frequency, and pulse width are standardised to zero mean and unit variance using the training-set mean and standard deviation of each field, so that channels measured in different physical units contribute comparably to dot products in the encoder [12]. Each TOA is first

min–max scaled to $[0, 1]$ using training-set bounds; within every window, all rescaled TOA values are shifted by subtracting the TOA of the first pulse so that the window starts at time zero. This removes absolute clock offset while preserving intra-window timing structure.

4.2.5 Augmentations for contrastive views

Contrastive objectives in Section 4.4 consume two stochastic views of each training window. Those views combine the deterministic normalisation above with additional pulse-level or window-level perturbations (for example timing jitter, amplitude scaling, small frequency offsets, or pulse dropout) so that invariances reflect both simulator noise and plausible analogue front-end variability. The exact augmentation set and sampling ranges are part of the training configuration and are listed with other hyperparameters in Section 5.3.

Windows are grouped into mini-batches for optimisation; batch size and per-epoch shuffling of windows are stated in Section 5.3.

4.3 Transformer Encoder Architecture

The encoder f_{θ} takes the form of a transformer-encoder backbone [8] with a shared trunk followed by two task-specific branches; the overall layout is shown in Figure 4.2. This structure mirrors the dual-embedding formulation introduced in Section 4.1: the trunk produces a single contextualised representation of the input window, which the branches specialise into emitter-oriented and mode-oriented embeddings.

Input embedding

Each PDW vector $\mathbf{x}_n \in \mathbb{R}^d$, normalised as described in Section 4.2, is mapped to a token embedding through an affine projection

$$\mathbf{h}_n^{(0)} = \mathbf{W}_{\text{in}} \mathbf{x}_n + \mathbf{b}_{\text{in}}, \quad \mathbf{h}_n^{(0)} \in \mathbb{R}^{d_{\text{model}}}, \quad (4.7)$$

where $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d_{\text{model}} \times d}$ and d_{model} is the trunk width specified below. No positional encoding is added to $\mathbf{h}_n^{(0)}$. The temporal ordering of pulses is already carried by the TOA entry of $\mathbf{x}_n$, which makes a separate position signal redundant; consistent with this argument, Gunn et al. [14] report that adding sinusoidal or learnable positional encodings to a transformer deinterleaver yields a small but reproducible drop in performance, attributed to the resulting redundancy with the TOA feature.

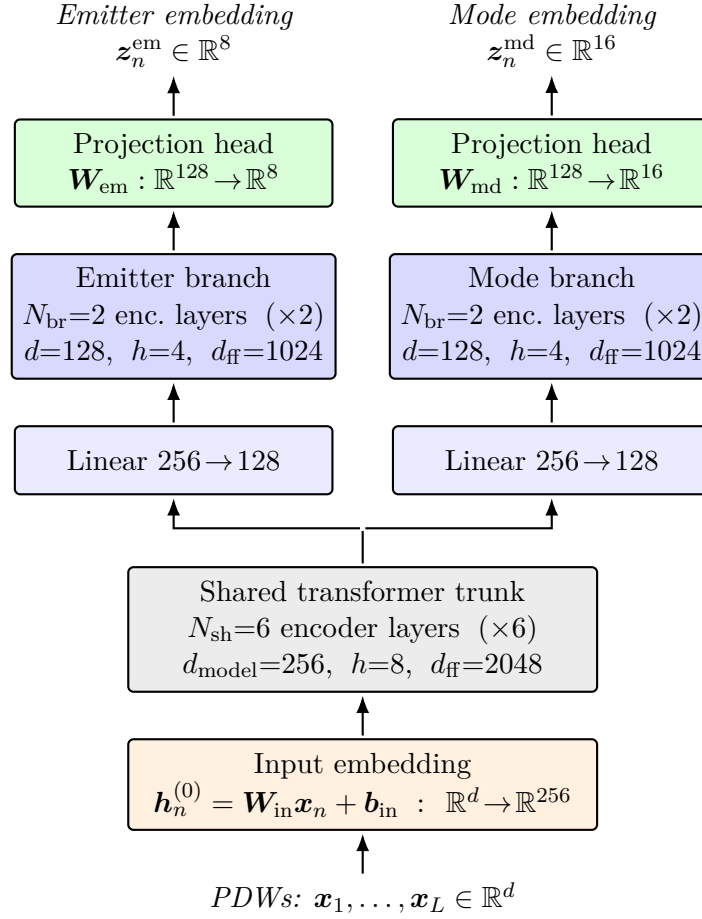


Figure 4.2. Architecture of the encoder f_{θ} . A sequence of PDWs is first lifted to $d_{\text{model}} = 256$ by an input embedding, processed by a shared trunk of 6 transformer-encoder layers, and then split into two parallel branches with their own $256 \rightarrow 128$ projection and 2 encoder layers. Branch outputs are mapped by linear projection heads to the emitter embedding space $\mathbb{R}^8$ and the mode embedding space $\mathbb{R}^{16}$.

Shared trunk

The token sequence $(\mathbf{h}_1^{(0)}, \dots, \mathbf{h}_L^{(0)})$ is processed by a stack of N_{sh} standard transformer-encoder layers, each comprising a MHA sub-layer and a position-wise FFN sub-layer with residual connections and layer normalisation around both. Self-attention is unmasked, so every pulse can attend to every other pulse in the window. The trunk operates with hidden dimension $d_{\text{model}} = 256$, $h_{\text{sh}} = 8$ attention heads (so that each head has dimension 32), and an inner FFN dimension of $d_{\text{ff,sh}} = 2048$; $N_{\text{sh}} = 6$ layers are used. Its output is a contextualised token sequence $\mathbf{H}^{\text{sh}} = (\mathbf{h}_1^{\text{sh}}, \dots, \mathbf{h}_L^{\text{sh}})$ with $\mathbf{h}_n^{\text{sh}} \in \mathbb{R}^{256}$, in which information has been mixed across the full window.

Task-specific branches

The trunk output feeds two parallel branches that do not share parameters, producing the dual embeddings of Equation (4.6). Each branch begins with an affine projection that reduces the hidden width from $d_{\text{model}} = 256$ to $d_{\text{br}} = 128$, after which $N_{\text{br}} = 2$ transformer-encoder layers of the same form as the trunk are applied. Within a branch, MHA uses $h_{\text{br}} = 4$ heads (again with per-head dimension 32) and the FFN inner dimension is $d_{\text{ff,br}} = 1024$. The reduced width and depth reflect a deliberate asymmetry: the trunk is intended to carry the heavy lifting of contextual mixing, while the branches only need to specialise the representation to either emitter identity or operating mode. Write the resulting token sequences as $\mathbf{H}^{\text{em}} = (\mathbf{h}_1^{\text{em}}, \dots, \mathbf{h}_L^{\text{em}})$ and $\mathbf{H}^{\text{md}} = (\mathbf{h}_1^{\text{md}}, \dots, \mathbf{h}_L^{\text{md}})$ with $\mathbf{h}_n^{\text{em}}, \mathbf{h}_n^{\text{md}} \in \mathbb{R}^{128}$.

Projection heads

Each branch terminates in a linear projection head that maps every token to its respective embedding space,

$$\mathbf{z}_n^{\text{em}} = \mathbf{W}_{\text{em}} \mathbf{h}_n^{\text{em}} + \mathbf{b}_{\text{em}}, \quad \mathbf{z}_n^{\text{md}} = \mathbf{W}_{\text{md}} \mathbf{h}_n^{\text{md}} + \mathbf{b}_{\text{md}}, \quad (4.8)$$

with $\mathbf{z}_n^{\text{em}} \in \mathbb{R}^p$ and $\mathbf{z}_n^{\text{md}} \in \mathbb{R}^q$ for $p = 8$ and $q = 16$. The mode space is assigned the higher dimensionality because, conditional on emitter identity, mode structure is expected to require a richer geometry to separate subtle waveform regimes, whereas emitter membership induces a coarser partition over the same pulses and is well served by a more compact embedding.

Per-window clustering with HDBSCAN

The projection heads in Equation (4.8) map each pulse to a continuous vector, whereas the latent emitter and mode assignments in Section 4.1 are discrete. HDBSCAN [15, 16] is applied independently to the two embedding trajectories within every analysis window: the L points $\{\mathbf{z}_n^{\text{em}}\}_{n=1}^L$ and $\{\mathbf{z}_n^{\text{md}}\}_{n=1}^L$ each form a separate clustering instance

on the corresponding branch. Each window is thus a self-contained pulse train segment in embedding space, consistent with the windowing of Section 4.2, and the effective cardinalities of the inferred emitter and mode partitions follow the density hierarchy instead of fixing K_{em} or K_{md} a priori.

As defined in the framework of Campello et al. [15], HDBSCAN replaces the single global density threshold of DBSCAN [4] with a hierarchy of density estimates: nested level sets are summarised in a condensed tree, and a flat clustering is read off by favouring groups that persist as coherent peaks rather than artefacts of a particular threshold. The same construction supports an explicit noise category for points that are not assigned to any cluster leaf retained after the hierarchy is flattened [15]. The computations use the reference library described by McInnes et al. [16].

The collection of all linear maps in Equations (4.7) and (4.8) together with the attention and FFN weights of the trunk and branches constitutes the parameter vector θ that the loss in Section 4.4 optimises.

4.4 Loss Function and Clustering Objective

The trainable objective combines contrastive terms on the emitter and mode branches so that both embedding geometries in Equation (4.6) become informative for the unsupervised clusterings described in Section 4.1. This section summarizes the representation-learning rationale, states a generic normalized temperature-scaled cross-entropy (NT-Xent) building block [6], and explains how it is instantiated separately for emitter-centric deinterleaving and for mode-centric structure.

Contrastive learning viewpoint

Contrastive objectives encourage invariance to nuisance transformations while preserving factors that vary across instances [6]. Given two stochastic *views* of the same underlying input (for example two augmentations of the same PDW window), the model should assign similar embeddings to both views, whereas embeddings of unrelated inputs should disagree. Minimizing the resulting classification-style loss is closely related to maximizing a lower bound on mutual information between representations and predictive targets in contrastive predictive coding [7], and more pragmatically it acts as a metric-learning surrogate that spreads the batch on the hypersphere defined by ℓ_2 normalization. A large set of negatives drawn from the mini-batch is essential: without repulsive pressure on unrelated pairs, encoders can trivially collapse to a constant map and still achieve low training error on a naïve agreement loss [6].

Swapping cluster assignments instead of raw embeddings is an alternative that couples representation learning with discrete structure [11]; the present work stays with embedding-space contrast for clarity of exposition, but the same dual-embedding architecture could be paired with a SwAV-style term if prototype banks were introduced.

NT-Xent building block

Let $\phi(\mathbf{z}) = \mathbf{z}/\|\mathbf{z}\|_2$ denote ℓ_2 normalization and let $\tau > 0$ denote the contrastive temperature. Write the scaled cosine similarity as

$$s_\tau(\mathbf{a}, \mathbf{b}) = \frac{1}{\tau} \phi(\mathbf{a})^\top \phi(\mathbf{b}). \quad (4.9)$$

The temperature τ rescales inner products after normalization: smaller τ sharpens the softmax in Equation (4.10), so the loss concentrates on the hardest negatives and gradients emphasize fine separation on the sphere, whereas larger τ yields a softer distribution closer to uniform and can stabilize optimization when embeddings are poorly separated early in training [6]. All contrastive terms in this thesis share a fixed $\tau = 0.2$, in line with values often used for NT-Xent on normalized features [6].

For a finite index set $\mathcal{I}$ with embeddings $\{\mathbf{z}_i\}_{i \in \mathcal{I}}$ and a distinguished *positive* index $p(i) \neq i$ for each anchor $i \in \mathcal{I}$, define the InfoNCE / NT-Xent contribution

$$\mathcal{L}_{\text{NT}}(\mathcal{I}, \{p(i)\}_{i \in \mathcal{I}}) = -\frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \log \frac{\exp(s_\tau(\mathbf{z}_i, \mathbf{z}_{p(i)}))}{\sum_{j \in \mathcal{I}, j \neq i} \exp(s_\tau(\mathbf{z}_i, \mathbf{z}_j))}. \quad (4.10)$$

The denominator collects every candidate in the mini-batch except the anchor itself, so unrelated pairs act as negatives and collapse is discouraged [6]. The same expression is reused below with different choices of index sets, corresponding to different geometric constraints on the emitter and mode spaces.

Emitter-oriented term (deinterleaving)

The emitter branch targets a representation in which pulses from the same physical emitter are close and pulses from different emitters are separated, which is the geometric content expected of a deinterleaving embedding [14]. For each window in a mini-batch, two views are formed by independent draws of the augmentation pipeline (once augmentations beyond deterministic scaling are fixed in Section 4.2); the two views share semantics at the pulse level up to controlled perturbations, so they play the role of a positive pair in the sense of SimCLR [6].

Because emitter identity is a *window-level* hypothesis in dense traffic, the contrastive signal is applied to a per-window summary of the emitter tokens rather than to isolated pulses without batch context. Let $b = 1, \dots, B$ index windows in a batch and let $\bar{\mathbf{z}}_b^{\text{em}} \in \mathbb{R}^p$ denote a fixed aggregation of $\{\mathbf{z}_{b,n}^{\text{em}}\}_{n=1}^L$, such as the arithmetic mean over pulse positions after Equation (4.8). For each b , collect the two pooled view embeddings $\bar{\mathbf{z}}_{b,(1)}^{\text{em}}$ and $\bar{\mathbf{z}}_{b,(2)}^{\text{em}}$ and form the index set $\mathcal{I}_b^{\text{em}} = \{(b, 1), (b, 2)\}$ together with the positive pairing that swaps the two views. Concatenate negatives by including the first-view pooled embeddings of all *other* windows in the batch, following the standard “one positive, many negatives” recipe [6]. The emitter loss is the batch average of per-window

NT-Xent values,

$$\mathcal{L}_{\text{em}} = \frac{1}{B} \sum_{b=1}^B \mathcal{L}_{\text{NT}}(\mathcal{I}_b^{\text{em}} \cup \mathcal{N}_b^{\text{em}}, p(\cdot)). \quad (4.11)$$

where $\mathcal{N}_b^{\text{em}}$ denotes the chosen negative multiset for window b . This mean over windows matches the implementation choice to score each window’s contrastive problem separately before aggregating, which keeps the repulsive set size stable when B changes.

Mode-oriented term

Operating modes induce finer structure within an emitter, often visible only when comparing pulse patterns across longer horizons than a single position index. The mode branch therefore uses the same NT-Xent functional form but mixes *intra-window* comparisons with *inter-window* comparisons so gradients flow both along the token sequence and between windows drawn from the same batch.

Intra-window mode contrast follows Equation (4.10) with $\mathcal{I}$ equal to pulse indices inside a window, positives given by the second augmented view at the same token index, and negatives taken from other positions in the same window and view, which encourages local discrimination of mode-relevant features.

Inter-window mode contrast forms ordered pairs (b, b') of distinct batch indices (for example random pairs or pairs chosen to exploit the half-window stride in Section 4.2 when two windows overlap in time). For each pair, embeddings $\mathbf{z}_{b,n}^{\text{md}}$ and $\mathbf{z}_{b',n'}^{\text{md}}$ are compared for a small set of index pairs (n, n') (including the diagonal $n = n'$ if both windows are aligned to the same clock after preprocessing). The resulting NT-Xent terms are averaged over sampled pairs and, together with the intra-window term, define $\mathcal{L}_{\text{md}}$. The construction parallels visual pipelines in which two “crops” of the same image are positives [6], except that the natural pairing is induced by overlapping windows and augmentation rather than by spatial cropping.

Combined objective and use of the projection heads

The overall training loss is the weighted sum

$$\mathcal{L} = \mathcal{L}_{\text{em}} + \lambda_{\text{md}} \mathcal{L}_{\text{md}}, \quad \lambda_{\text{md}} > 0, \quad (4.12)$$

with λ_{md} treated as a hyperparameter and reported in Chapter 5. An auxiliary DEC-style KL term between soft cluster assignments and a target distribution [10] is a natural extension when trainable centroids or prototypes are added; the experiments chapter states whether such a term is active.

Following common practice in self-supervised encoders [6], the mode embeddings retained for visualization or nearest-neighbor mode analysis are the vectors $\mathbf{z}_n^{\text{md}}$ from Equation (4.8), that is, the outputs of the mode linear head. For emitter-centric deinterleaving, the downstream deliverable is primarily the discrete per-pulse assignment

$\hat{e}_n$ obtained by clustering pooled emitter embeddings or by reading off a discrete head if one is added later; once those labels are fixed, storing every $\mathbf{z}_n^{\text{em}}$ is optional because the continuous representation is not required for the same operational tasks that consume hard emitter tracks.

4.5 Training Procedure

The parameter vector $\boldsymbol{\theta}$ attached to the maps in Equations (4.7) and (4.8) is estimated by minimising the training objective $\mathcal{L}$ in Equation (4.12) on mini-batches of windows. Gradients are obtained by reverse-mode automatic differentiation and used in an adaptive first-order optimiser of the Adam family [17]. Unless Section 5.3 specifies otherwise, the implementation follows the default moment decay rates $(\beta_1, \beta_2) = (0.9, 0.999)$, stabiliser $\epsilon = 10^{-8}$, and bias-corrected running estimates of the first and second moments described by Kingma and Ba [17], as exposed by the PyTorch optimiser interface.

The learning rate follows a cosine decay from a peak value down to a small floor over the course of training, which keeps step sizes comparatively large while the loss landscape is coarse and shrinks them during the final refinement phase. Peak learning rate, weight decay, batch size, number of passes over the training windows, and the exact step counting convention (per epoch versus global step) are hyperparameters and are listed in Table 5.2. At the start of each epoch, training windows are shuffled so that consecutive mini-batches are not ordered by scenario; any fixed random seeds, data-loader worker counts, and reproducibility settings used in the reported runs are stated in Sections 5.1 and 5.3.

Dropout inside the transformer blocks of Section 4.3 follows the usual pattern of stochastic attention and feed-forward masks; dropout rates belong to the same hyperparameter table. Training can be stopped early when a validation score computed on held-out windows (for example the smoothed mini-batch loss or an auxiliary clustering criterion) fails to improve for a fixed patience measured in epochs, in which case the best checkpoint prior to stagnation is retained; patience, validation metric, and checkpoint policy are also recorded in Table 5.2.

If a DEC-style Kullback–Leibler term is added as suggested in Section 4.4, target distributions are recomputed from the student assignments on a schedule analogous to Xie et al. [10], and any sharpening temperature or update period is treated as part of the optimisation protocol reported in Section 5.3. Likewise, the mode-branch weight λ_{md} may be held small during an initial warm-up segment so that the trunk learns coarse temporal structure before the finer mode contrastive gradients dominate; whether such a ramp is used, and its length, is part of the same configuration table. Multi-phase schedules, such as representation pretraining followed by clustering fine-tuning, are compatible with the same tooling; if they are employed, the phases and their objectives are stated in Section 5.3.

4.6 Evaluation Metrics

Let y_n and $\hat{y}_n$ denote reference and predicted cluster indices for pulse $n \in \{1, \dots, N\}$, as in Equations (4.2) and (4.6), instantiated by $(e_n, \hat{e}_n)$ or $(m_n, \hat{m}_n)$. The contingency is $n_{ij} = \sum_{n=1}^N \mathbf{1}[y_n = i, \hat{y}_n = j]$ with marginals $a_i = \sum_j n_{ij}$ and $b_j = \sum_i n_{ij}$.

Adjusted Rand index

The ARI [18] is

$$\text{ARI} = \frac{\sum_{i,j} \binom{n_{ij}}{2} - [\sum_i \binom{a_i}{2}][\sum_j \binom{b_j}{2}] / \binom{N}{2}}{\frac{1}{2} [\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}] - [\sum_i \binom{a_i}{2}][\sum_j \binom{b_j}{2}] / \binom{N}{2}}, \quad \text{ARI} \leq 1. \quad (4.13)$$

Perfect agreement up to relabelling gives $\text{ARI} = 1$.

Entropies, mutual information, and normalizations

Shannon entropies of the marginals are

$$H(Y) = - \sum_{i: a_i > 0} \frac{a_i}{N} \log \frac{a_i}{N}, \quad H(\hat{Y}) = - \sum_{j: b_j > 0} \frac{b_j}{N} \log \frac{b_j}{N}. \quad (4.14)$$

The empirical mutual information is [19]

$$\text{MI}(Y; \hat{Y}) = \sum_{i,j: n_{ij} > 0} \frac{n_{ij}}{N} \log \frac{N n_{ij}}{a_i b_j}, \quad (4.15)$$

with $0 \log 0 = 0$. A NMI score is obtained by dividing MI by a function of $\{H(Y), H(\hat{Y})\}$; we use the arithmetic-mean denominator [19],

$$\text{NMI}(Y; \hat{Y}) = \frac{2 \text{MI}(Y; \hat{Y})}{H(Y) + H(\hat{Y})}, \quad H(Y) + H(\hat{Y}) > 0, \quad (4.16)$$

and set $\text{NMI} = 0$ if $H(Y) + H(\hat{Y}) = 0$. Let $\mathbb{E}[\text{MI}]$ be the expectation of MI under the permutation null that fixes one partition while permuting labels subject to fixed cluster sizes [19]. The AMI is

$$\text{AMI}(Y; \hat{Y}) = \frac{\text{MI}(Y; \hat{Y}) - \mathbb{E}[\text{MI}]}{\frac{1}{2}(H(Y) + H(\hat{Y})) - \mathbb{E}[\text{MI}]}, \quad (4.17)$$

when the denominator is positive; otherwise $\text{AMI} \equiv 0$.

Clustering accuracy

Cluster indices are arbitrary up to a bijection ϕ on label symbols. The ACC is the largest agreement after an optimal relabeling,

$$\text{ACC} = \frac{1}{N} \max_{\phi \in \Phi} \sum_{n=1}^N \mathbf{1}[y_n = \phi(\hat{y}_n)], \quad (4.18)$$

where Φ is the set of bijections between the active reference labels and the active predicted labels when their counts coincide; if cardinalities differ, the same maximum is computed by the Hungarian (linear-sum assignment) algorithm on the matrix (n_{ij}) with cost $-n_{ij}$ [20].

V-measure

With conditional entropies

$$H(Y | \hat{Y}) = - \sum_{i,j: n_{ij} > 0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{b_j}, \quad H(\hat{Y} | Y) = - \sum_{i,j: n_{ij} > 0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{a_i}, \quad (4.19)$$

homogeneity and completeness are [21]

$$h = 1 - \frac{H(Y | \hat{Y})}{H(Y)}, \quad c = 1 - \frac{H(\hat{Y} | Y)}{H(\hat{Y})}, \quad (4.20)$$

with $h = 1$ if $H(Y) = 0$ and $c = 1$ if $H(\hat{Y}) = 0$. The V-measure is

$$V(Y, \hat{Y}) = \frac{2hc}{h+c}, \quad h+c > 0, \quad (4.21)$$

and $V = 0$ if $h = c = 0$.

Figures

Two-dimensional projections of $\{\mathbf{z}_n^{\text{em}}\}$ and $\{\mathbf{z}_n^{\text{md}}\}$ coloured by (e_n, m_n) complement Equations (4.13), (4.16) to (4.18) and (4.21); protocols and numbers appear in Chapter 5.

4.7 Baseline Methods

The baselines isolate the contribution of the dual-branch layout in Section 4.3 and of the mode-oriented term $\mathcal{L}_{\text{md}}$ in Equation (4.12) relative to a single-stack encoder and to classical clustering without a sequence model.

The first neural baseline removes the emitter and mode branches and replaces them with one stack of eight transformer-encoder layers of the same functional form as the

shared trunk (hidden width, head count, and FFN expansion match that trunk so the comparison stresses topology and objective rather than a change in per-layer width). Token embeddings $\mathbf{h}_n^{(0)}$ from Equation (4.7) pass through this stack; a single linear head then maps each contextualised token to $\mathbf{z}_n \in \mathbb{R}^p$, using the same output dimension p as the emitter branch in the full model. Training minimises only the emitter-oriented contrastive loss $\mathcal{L}_{\text{em}}$ in Equation (4.11); $\mathcal{L}_{\text{md}}$ and its inter-window structure are omitted, so no auxiliary geometry is trained for operating modes. This configuration tests whether one representation, optimised purely for window-level deinterleaving, suffices for both emitter- and mode-level clusterings reported in Chapter 5.

The second baseline is HDBSCAN [16], a hierarchical density-based method that does not estimate a transformer. It clusters fixed-length vectors built deterministically from the per-feature scaled PDW windows in Section 4.2, for example by per-channel summary statistics or a flattened layout of pulses and features; the exact vectorisation is fixed in Chapter 5. HDBSCAN separates gains due to learned sequence context from what density-based Euclidean clustering already recovers on hand-specified summaries.

Further comparisons— k -means on raw window vectors or on autoencoder embeddings, DEC-style objectives when centroids are used, SwAV-style assignment prediction [11], and a supervised classifier upper bound when labels are available—are listed with their configurations in Chapter 5.

CHAPTER 5

Experiments and Results

5.1 Experimental Setup

All neural models in this chapter were trained and evaluated on a single workstation with one NVIDIA A100 GPU (80 GB high-bandwidth memory; some system utilities report a capacity closer to 82 GB because of binary versus decimal gigabyte conventions). The implementation uses Python with PyTorch; exact package versions are pinned in the environment file shipped with the thesis code repository when that repository is published.

Random seeds control weight initialization, training-window shuffling each epoch, and stochastic augmentations. Unless an experiment explicitly varies the seed, the same seed set is reused so that ablations in Section 5.5 are comparable under identical noise conditions. Concrete seed integers, per-run overrides, and mean $\pm$ standard deviation over multiple seeds for quantitative scores are reported with the tables in Section 5.4.

5.2 Datasets

All experiments in this chapter use synthetic PDW streams generated by the proprietary scenario simulator introduced in Section 4.2. The simulator supplies time-ordered pulse records together with ground-truth emitter identity and operating mode for each pulse; those labels are withheld from the training objective and are used only to compute the clustering metrics in Sections 4.6 and 5.4. When a run includes receiver-side degradations (dropped or spurious pulses), they are applied to the ordered pulse lists as in Section 4.2.2 and Algorithm 1 before windowing, and the tables below state which regimes are active. Operational ELINT recordings are not included, which sidesteps distribution shift and disclosure constraints while still exercising mixture traffic, mode changes within emitters, and variable train density.

Each *scenario* is one simulated timeline: a sequence of PDWs whose length and the number of concurrent emitters vary across draws so that the encoder sees both sparse and crowded electromagnetic pictures. Scenarios are disjointly assigned to training, validation, and test partitions before any windowing; all normalisation statistics in Section 4.2 are fit on the training partition only and applied unchanged to validation and test pulses. Training windows use stride $L/2$ with $L = 1024$ pulses, whereas validation and test windows use stride L , so the effective number of windows per scenario differs by split even when raw pulse counts are similar.

Table 5.1 summarises corpus-level figures. Emitter and mode columns count distinct

Table 5.1. Synthetic scenario corpus used for training and evaluation. Dashes mark quantities still being reconciled with the export manifest.

Split	Scenarios	Pulses (approx.)	Distinct emitters	Distinct modes
Train	—	—	—	—
Validation	—	—	—	—
Test	—	—	—	—
All	—	—	—	—

simulator labels that appear anywhere in the corresponding split; they are upper bounds on the cardinality that unsupervised clustering must recover, because not every label may be present in every window. Scenario design and traffic rules also skew how often each emitter and each mode is exercised, so aggregate pulse counts need not match intuitive “balanced” priors even when the simulator exposes many labels. Because the encoder sees only windows of fixed length L (Section 4.2), rare combinations of emitter, mode, and interleaving pattern can be missing from individual windows or represented by only a handful of pulses, which limits what any single forward pass can evidence about those labels. The numeric entries are placeholders until the final export from the simulator pipeline is frozen; the intended reporting convention is scenarios for volume, pulses for aggregate exposure, and distinct labels for emitter and mode coverage.

5.2.1 Qualitative properties of the synthetic corpus

Beyond aggregate counts, the corpus is checked qualitatively before large training runs: example timelines are inspected for plausible PRI structure per mode, for physically admissible RF and pulse-width ranges, and for the intended mixture density when multiple emitters are active. When receiver-side degradations are enabled (Section 4.2.2), spot checks confirm that dropped pulses thin pulse trains without reordering surviving pulses and that spurious insertions remain within the configured rule set. This mirrors the reporting practice in earlier Saab-hosted simulator studies, which often include illustrative emitter tracks alongside tables [12]. Section 5.6 complements the brief sanity checks here with embedding-space visualizations and other qualitative diagnostics once model checkpoints are available.

5.3 Hyperparameters and Training Details

This section collects numerical training choices deferred from Sections 4.3 to 4.5. Unless Section 5.5 states otherwise, baseline models use the same epoch budget, hardware (Section 5.1), and early-stopping protocol so that differences reflect architecture and objective rather than compute.

The contrastive temperature is fixed at $\tau = 0.2$ as in Section 4.4. Table 5.2

Table 5.2. Primary optimisation hyperparameters for the proposed model.

Setting	Value
Optimiser	Adam [17] with PyTorch defaults (β_1, β_2) = (0.9, 0.999) and $\epsilon = 10^{-8}$
Peak learning rate $\eta_{\max}$	—
Minimum learning rate $\eta_{\min}$	0 (cosine floor; PyTorch default)
Cosine period $T_{\max}$	30 scheduler steps, one step per training epoch
Maximum epochs	30
Early stopping patience	7 epochs without improvement on the validation loss
Early stopping checkpoint	Weights from the epoch with lowest validation loss so far
Dropout (attention and feed-forward)	0.1 throughout the transformer blocks
Batch size B (windows per step)	—
Mode loss weight λ_{md}	—
Weight decay	—

summarises the optimisation schedule and regularisation for the proposed dual-branch encoder; entries marked with an em dash are still being matched to the released training script and will be filled before the final thesis submission.

The cosine scheduler follows the PyTorch `CosineAnnealingLR` convention: after each epoch the learning rate is updated from $\eta_{\max}$ toward $\eta_{\min}$ over $T_{\max} = 30$ steps, which aligns the period with the nominal training horizon. If early stopping triggers first, training halts at the best validation checkpoint and the final few scheduler steps may not be executed.

5.4 Quantitative Results

This section reports primary clustering scores for the proposed model and for each baseline under the data conditions in Section 5.2. Rows index methods (including ablated variants where applicable); columns index emitter-level and mode-level NMI, ARI, and ACC after optimal alignment of cluster indices to reference labels where those labels exist only for evaluation (Section 4.6). Each entry is aggregated over random seeds and window splits as stated in Sections 5.1 and 5.3; mean $\pm$ one standard deviation is shown when multiple seeds are available.

5.5 Ablation Studies

Ablations isolate the contribution of individual components in Chapter 4 by retraining with exactly one change relative to the full configuration (matched data, seeds, and

hyperparameters unless the ablation itself is a hyperparameter). Planned comparisons include removing or downweighting the emitter contrastive term, the mode contrastive term, or the coupling between branches; reducing the number of attention layers or heads; disabling stochastic augmentations beyond deterministic scaling; and swapping HDBSCAN post-processing for a simpler fixed- k assignment when that variant is implemented.

5.6 Qualitative Analysis

Qualitative analysis complements the scalar metrics in Section 5.4 by checking whether embeddings organize pulses in ways that align with physical emitter boundaries and with mode transitions inside a window. The checks in Section 5.2.1 validate that the simulator export behaves as intended; this section applies analogous scrutiny *after* models are trained.

Two-dimensional projections of pooled or per-pulse embeddings (for example UMAP or t-SNE) should be colored separately by reference emitter and by reference mode when labels exist only for evaluation. Disjoint emitter-colored manifolds with mode-colored substructure that stays coherent within an emitter are evidence that the dual geometry in Chapter 4 is behaving as designed; elongated bridges between emitter manifolds often indicate residual deinterleaving ambiguity or shared waveform parameters across platforms.

Attention maps or head-wise statistics, if exported, can be overlaid on TOA-ordered pulses to see whether specific heads track stable intervals versus rapid mode hops.

Confusion matrices between inferred clusters and reference labels (after optimal assignment) are optional but useful when a small number of emitter or mode classes dominate error mass.

CHAPTER 6

Discussion

6.1 Interpretation of Results

This section connects the quantitative tables in Section 5.4 and the qualitative material in Section 5.6 to the modeling choices in Chapter 4. The goal is not to restate individual scores, but to explain *which* mechanisms plausibly drive the observed ranking of methods and ablations.

When emitter-level NMI or ARI lag while mode-level scores are stronger, a natural hypothesis is that the encoder prioritizes intra-window structure tied to waveform parameters (which modes share) before it fully exploits slower cues that separate physical platforms across mixed traffic. Conversely, if emitter scores dominate but mode partitions collapse, the dual-head coupling or the mode contrastive term may be underweighted relative to the emitter branch, or certain modes may be too short-lived inside length- L windows to support stable clusters.

Noise and spurious-pulse regimes from Section 4.2.2 interact with self-attention in two ways: they perturb local timing patterns that define modes, and they change which pulses co-occur inside a window, which can confuse any window-level emitter summary. Interpretation should therefore read noise sweeps alongside the corpus diagnostics in Sections 5.2.1 and 5.6: a metric drop that coincides with visibly corrupted tracks is evidence of a robustness limit, whereas a drop on clean held-out scenarios points toward generalization or optimization issues instead.

6.2 Comparison with Related Work

Relative to classical ELINT fingerprinter pipelines surveyed in Section 3.1, the proposed approach trades hand-engineered feature logic for a learned encoder, at the cost of data hunger, compute, and reduced transparency unless accompanied by diagnostics such as those in Section 5.6. Unlike supervised radar classifiers discussed in Section 3.2, the training objective here does not assume a fixed label set for emitters or modes at train time; evaluation still uses reference labels where available, but they are not required by the learning rule itself.

Deep clustering methods in Section 3.3 typically optimize a *single* flat partition or require labels for each head. This thesis instead targets two aligned partitions with a nesting interpretation (modes within emitters), which is closer in spirit to offline pulse-level grouping work that reasons about geometry and overlap in ESM data [22] but uses gradient-based representation learning rather than hand-tuned density stages

alone. Where scores fall short of the best published supervised benchmarks, the gap is expected: those models exploit label information throughout training and often operate on curated single-emitter tracks, whereas the present setting stresses mixture windows and unsupervised objectives.

6.3 Limitations

The experimental programme is offline: models are trained and scored on bounded, revisit-able windows rather than under the latency, memory, and non-stationarity constraints of a continuously arriving pulse stream (Sections 1.4, 1.4.2 and 4.2). That setting supports controlled optimization and reproducible metrics but does not demonstrate that the same representations would support stable online emitter or mode tracking without periodic retraining or revised windowing policy.

Fixed-length windows are partly a consequence of full self-attention, whose standard implementation scales quadratically with the number of pulses in the window (Section 4.2). The chosen cap on L therefore trades faithful coverage of long mode segments and rare emitters against compute, and the present pipeline drops trailing segments shorter than L , which can remove evidence entirely when a scenario does not fill an integral number of windows.

Interleaved emitters and mode-hopping further skew how many pulses from each ground-truth label appear inside any given window, so performance summaries aggregate over heterogeneous label incidence rather than over an idealized balanced design (Section 5.2). Where metrics depend on comparing inferred partitions to reference labels, conclusions are most informative for label configurations that actually recur with sufficient mass inside windows; weak support for a label in a window is a limitation of the observation model, not necessarily of the encoder in isolation.

Finally, all quantitative evidence is derived from synthetic scenarios with full reference labels; operational recordings with imperfect deinterleaving, sensor dropouts, and distribution shift are not included here. Related Saab-line work on track association under deinterleaving errors highlights that real pipelines fragment pulse trains and introduce structured mistakes that are only partly captured by the stylized drop and spurious models used for stress tests [23]. Likewise, classifiers trained on simulator draws can exhibit overconfidence on held-out emitters or modulation families, which motivates explicit out-of-distribution and drift analyses when moving toward fielded receivers [24]. The present thesis does not implement drift monitors or uncertainty heads; those remain orthogonal safeguards if embeddings from Chapter 4 are ever deployed under evolving threat libraries.

The HDBSCAN post-processing used in Section 4.3 also encodes density assumptions that may not match every deployment, and we do not exhaust efficient-attention or hierarchical alternatives that could extend context within the same hardware envelope.

6.4 Implications

For ELINT and ESM practice, the main takeaway is methodological: joint emitter- and mode-aware embeddings can in principle be learned without per-pulse supervision, but their reliability still hinges on scenario coverage, windowing policy, and honest reporting of failure modes under mixture and noise (Section 6.3). Operators should treat unsupervised cluster indices as hypotheses that require human or library-based adjudication before tactical use, especially when training data are synthetic and drift relative to live emitters is unmodeled.

For the deep-clustering and sequence-modeling research communities, the thesis sits at the intersection of contrastive representation learning on irregular pulse sequences and multi-partition clustering with a physical nesting constraint (modes within emitters). The architectural and loss choices in Chapter 4 are portable to other domains with hierarchical latent structure, provided that evaluation metrics separate which level of the hierarchy is being scored.

Follow-up work naturally includes streaming or sliding-window evaluation, richer deinterleaving and track-error models [23], and explicit ML safeguards against dataset shift and overconfident extrapolation [24]. On the ethics side, the considerations in Section 1.6 still apply: dual-use sensitivity, data stewardship if operational traces are introduced later, and the energy cost of transformer-scale training should be revisited whenever the experimental footprint grows.

CHAPTER 7

Conclusion

7.1 Summary

Placeholder paragraph.

7.2 Answers to the Research Questions

Placeholder paragraph.

7.3 Future Work

Placeholder paragraph.

References

- [1] R. G. Wiley, *ELINT: The Interception and Analysis of Radar Signals*. Artech House, 2006.
- [2] M. I. Skolnik, *Radar Handbook*, 3rd ed. McGraw-Hill, 2008.
- [3] Z. Qu, J. Zhang, Y. Zhou, and L. Ni, “The intelligent evolution of radar signal deinterleaving: A systematic review from foundational algorithms to cognitive AI frontiers”, *Sensors*, vol. 26, no. 1, 2026. DOI: 10.3390/s26010248
- [4] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise”, in *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.
- [5] Y. LeCun. “A path towards autonomous machine intelligence”. version 0.9.2, OpenReview, Accessed: May 13, 2026. [Online]. Available: <https://openreview.net/forum?id=BZ5a1r-kVsf>
- [6] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations”, in *International Conference on Machine Learning (ICML)*, 2020.
- [7] A. van den Oord, Y. Li, and O. Vinyals, *Representation learning with contrastive predictive coding*, arXiv preprint arXiv:1807.03748, 2018. arXiv: 1807.03748 [cs.LG].
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need”, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding”, in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.
- [10] J. Xie, R. Girshick, and A. Farhadi, “Unsupervised deep embedding for clustering analysis”, in *International Conference on Machine Learning (ICML)*, 2016, pp. 478–487.
- [11] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin, “Unsupervised learning of visual features by contrasting cluster assignments”, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [12] T. Jonsson, “Classification of Radar Emitters using Semi-Supervised Contrastive Learning”, Host company Saab AB; supervised learning experiments on pulse data from the OpenModesty scenario simulator, Master’s thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2023.

- [13] A. Svensson, “Classification of Radar Emitters Based on Pulse Repetition Interval using Machine Learning”, Host company Saab AB; synthetic PRI sequences from specification-driven generators with controlled noise regimes, Master’s thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2022.
- [14] E. Gunn, A. Hosford, D. Mannion, J. Williams, V. Chhabra, and V. Nockles, *Radar pulse deinterleaving with transformer based deep metric learning*, arXiv preprint arXiv:2503.13476, 2025. arXiv: 2503.13476 [cs.LG].
- [15] R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander, “Hierarchical density estimates for data clustering, visualization, and outlier detection”, *ACM Transactions on Knowledge Discovery from Data*, vol. 10, no. 1, 2015. DOI: 10.1145/2733381
- [16] L. McInnes, J. Healy, and S. Astels, “hdbscan: Hierarchical density based clustering”, *The Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017. DOI: 10.21105/joss.00205
- [17] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, arXiv preprint arXiv:1412.6980, 2017. arXiv: 1412.6980 [cs.LG].
- [18] L. Hubert and P. Arabie, “Comparing partitions”, *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.
- [19] N. X. Vinh, J. Epps, and J. Bailey, “Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance”, *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010.
- [20] H. W. Kuhn, “The Hungarian method for the assignment problem”, *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955. DOI: 10.1002/nav.3800020109
- [21] A. Rosenberg and J. Hirschberg, “V-measure: A conditional entropy-based external cluster evaluation measure”, in *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, 2007, pp. 410–420.
- [22] S. Bedoire, “Offline direction clustering of overlapping radar pulses from homogeneous emitters”, Host company Saab; DBSCAN-style clustering on pulse directions with simulated scenarios, Master’s thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2022.
- [23] V. Jakobsson, “Offline radar pulse track association with deinterleaving errors”, Host company SAAB; time-series clustering for track fragments under imperfect deinterleaving, Master’s thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2024.

-
- [24] K. Coleman, “Dataset drift in radar warning receivers: Out-of-distribution detection for radar emitter classification using an RNN-based deep ensemble”, UPTEC F 23041, 30 hp; simulator data from Saab AB and drift/OOD analysis for emitter classification, Master’s thesis, Uppsala University, Uppsala, Sweden, 2023.

APPENDIX A

Additional Results

Placeholder paragraph.

APPENDIX B

Implementation Details

Placeholder paragraph.

APPENDIX C

Notation Reference

Placeholder paragraph.